D01｜先查清失败发生在哪里

同一句“没查成”，可能需要补输入、检查外部服务或修内容合同。先沿 **本地输入 → 外部发送 → HTTP 状态 → JSON 与字段 → 本地回应** 找到最早停止处，再决定下一步。

有两次交换，哪一份回应到了？

当前问题	关系或短用法	记到什么程度
用户收到本地 504，说明合作方也回了 504 吗？	本地接口接收用户请求，又请求合作方；本地 response 与合作方 response 分别产生。示例超时中 `response = partner.get(...)` 未正常完成，本地仍能组织 504。	**认清两次交换和两份证据。** 真实流式读取可能已收到头部再超时，应保留实际收到的信息。
缺邮编、合作方 500、200 坏内容，分别查哪里？	缺邮编在本地校验处停，422、外部零调用；500 已有外部回应，本地转 502；200 后仍可能在 JSON 解析或字段合同处转 502。	按最早失败位置分类；状态码映射属于本例合同，不背成所有接口的固定规则。
JSON 能解析，为什么不能把缺失的 `serviceable` 当 False？	解析只取得 Python 值；本例还要求布尔 `serviceable` 和非空字符串 `zone`。默认 False 会把“无法判断”改成“确认不支持”。	**未知不能填成业务否定。** 先查顶层类型，再取字段和检查用途。
adapter 增加了什么？	构造外部请求、检查 status 与内容，把它们翻译成可用结果或本地失败类别；业务代码据此形成公开回应。	理解边界的责任；`raise_for_status()`、`.json()`、字段校验各负责一关。异常名可查。

`except TimeoutException` 放在更宽的 `RequestError` 前面，才能保留具体超时类别。接住异常只是形成明确失败，不会补出成功值；D02 再判断这类失败有没有理由再试。

回看：「一次查询里有两份不同的回复」「五种失败为何不能用一个“重试”处理？」「运行时最值得核对的一项变化」。

测试替换了哪里，就证明到哪里

当前问题	关系或短用法	记到什么程度
怎样可重复地制造坏 JSON 或超时？	HTTPX `MockTransport(handler)` 让本地函数接住真实构造的 request，返回预设 Response 或抛异常；adapter 仍执行。	认熟替换位置；主动抛超时不证明真实等待或网络接通。
TestClient 和有状态 fake 各做什么？	TestClient 经过本地路由、输入校验与回应生成，不监听端口；有状态 fake 保存业务效果、处理查询和重复调用，可与 MockTransport 配合。	按要观察的责任选工具，不把三者当成互相替代的名字。
直接把 adapter 换成成功字典，漏测什么？	只能看到外层包装，原来的请求参数、status 和字段检查没有运行。	**看被保留的代码与实际断言。** 预设成功不证明凭据或供应商可用。
输入拒绝时，只查没有新增效果够吗？	还要查 `calls == []`。本地 422、外部调用 1 次、效果 0，仍违反“先拒绝输入再执行”的边界。	逐渐用熟零调用断言；没有效果和没有调用分别看。

完整脚本与环境步骤查正文。实验在独立副本运行；这些进程内结果不覆盖 DNS、TLS、真实网络、实际端口或供应商合同。

回看：「让合作方按照指定剧本回答」「替换哪一处，决定测试证明到哪里」「运行时最值得核对的一项变化」。